Task 15: Implement Service Registry

Aim:

To implement a Service Registry and Discovery mechanism using Node.js that enables dynamic service lookup and eliminates hard-coded service endpoints.

Algorithm:

1. Initialize an in-memory registry object
2. Register services with name and URL
3. Store multiple instances per service
4. Use round-robin for load balancing
5. Provide discovery endpoint
6. Forward requests dynamically via API Gateway
7. Allow deregistration of services

Code:

Server.js

```
const express = require("express");
const bodyParser = require("body-parser");
const axios = require("axios");

const app = express();
app.use(bodyParser.json());

/* ============================
   HOME ROUTE (FIX ADDED)
============================ */
app.get("/", (req, res) => {
    res.send("Service Registry is Running 🚀");
});
```

```
/* ============================
   IN-MEMORY SERVICE REGISTRY
============================ */
let registry = {};
let loadBalancerIndex = {};

/* ============================
   1. REGISTER SERVICE
============================ */
app.post("/register", (req, res) => {
    const { serviceName, serviceUrl } = req.body;

    if (!serviceName || !serviceUrl) {
        return res.status(400).json({
            message: "serviceName and serviceUrl are required"
        });
    }

    if (!registry[serviceName]) {
        registry[serviceName] = [];
        loadBalancerIndex[serviceName] = 0;
    }

    // Avoid duplicate registration
    if (!registry[serviceName].includes(serviceUrl)) {
        registry[serviceName].push(serviceUrl);
    }
```

```
    res.json({
        message: `${serviceName} registered successfully`,
        services: registry[serviceName]
    });
});

/* ============================
   2. DISCOVER SERVICE
============================ */
app.get("/discover/:serviceName", (req, res) => {
    const serviceName = req.params.serviceName;

    const services = registry[serviceName];

    if (!services || services.length === 0) {
        return res.status(404).json({
            message: "Service not found"
        });
    }

    // Round Robin Load Balancing
    const index = loadBalancerIndex[serviceName] % services.length;
    const selectedService = services[index];

    loadBalancerIndex[serviceName]++;

    res.json({
        service: serviceName,
        url: selectedService
```

```
    });
});

/* ============================
   3. DEREGISTER SERVICE
============================ */
app.post("/deregister", (req, res) => {
    const { serviceName, serviceUrl } = req.body;

    if (!registry[serviceName]) {
        return res.status(404).json({
            message: "Service not found"
        });
    }

    registry[serviceName] = registry[serviceName].filter(
        url => url !== serviceUrl
    );

    res.json({
        message: "Service deregistered successfully",
        services: registry[serviceName]
    });
});

/* ============================
   4. API GATEWAY (CALL SERVICE)
============================ */
app.get("/call/:serviceName", async (req, res) => {
```

```
    const serviceName = req.params.serviceName;

    const services = registry[serviceName];

    if (!services || services.length === 0) {
        return res.status(404).json({
            message: "Service not found"
        });
    }

    const index = loadBalancerIndex[serviceName] % services.length;
    const selectedService = services[index];

    loadBalancerIndex[serviceName]++;

    try {
        const response = await axios.get(selectedService);

        res.json({
            calledService: selectedService,
            response: response.data
        });
    } catch (error) {
        res.status(500).json({
            message: "Error calling service",
            error: error.message
        });
    }
});
```

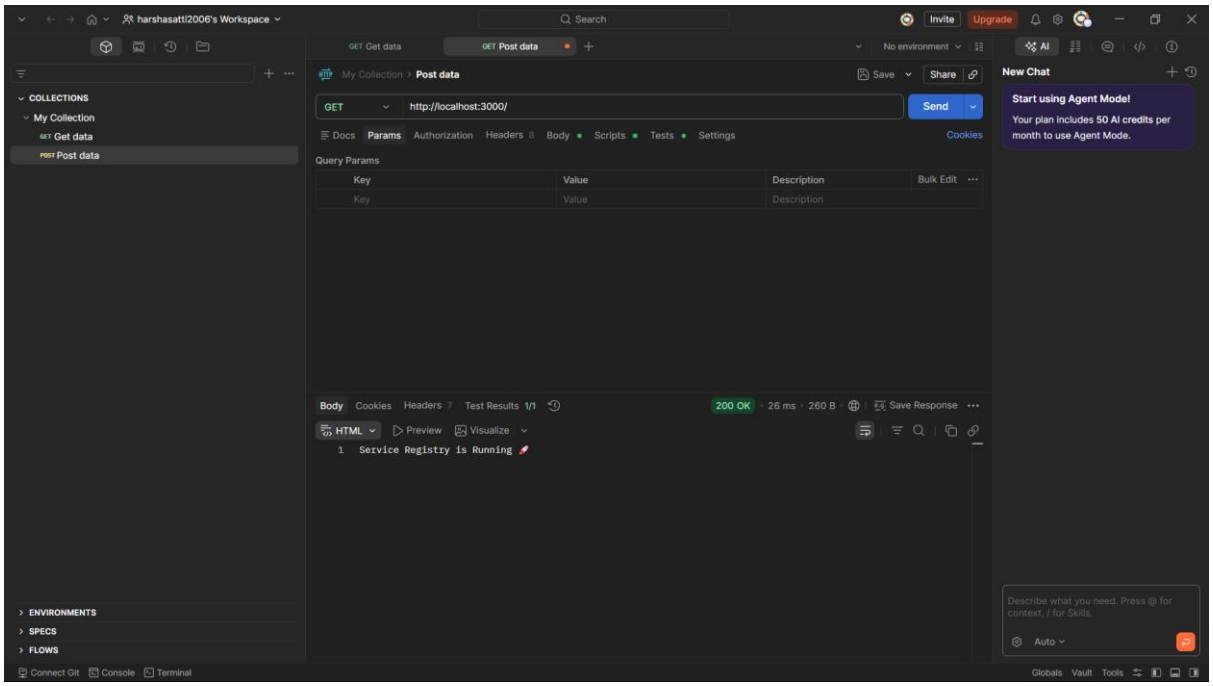
```
/* ============================
   SAMPLE MICROSERVICES
============================ */
app.get("/serviceA", (req, res) => {
    res.json({
        service: "A",
        message: "Response from Service A"
    });
});

app.get("/serviceB", (req, res) => {
    res.json({
        service: "B",
        message: "Response from Service B"
    });
});

/* ============================
   START SERVER
============================ */
const PORT = 3000;

app.listen(PORT, () => {
    console.log(`Service Registry running at http://localhost:${PORT}`);
});
```

Outputs:



Result: A working microservice registry system similar to Eureka is implemented using Node.js, supporting dynamic service registration, discovery, and load-balanced communication, successfully tested using Postman.